1. What is DBSCAN?

DBSCAN = Density-Based Spatial Clustering of Applications with Noise

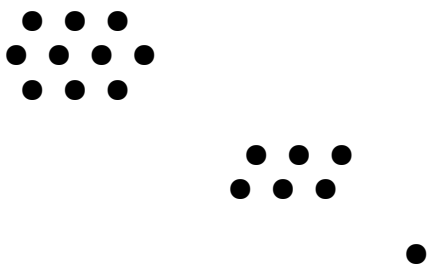
> DBSCAN groups data points that are closely packed together and identifies isolated points as noise/outliers.

It is a **density-based clustering algorithm**.

---

2. Main idea

Imagine points like this:



DBSCAN identifies:

```
First group  → Cluster 0
Second group → Cluster 1
Last point   → Noise
```

It doesn't require us to specify the number of clusters beforehand.

---

3. Two important parameters

DBSCAN mainly uses:

```
DBSCAN(eps=0.5, min_samples=5)
```

eps

The **maximum distance** used to find neighboring points.

> eps = How far should I look around a point?

min_samples

The **minimum number of points required within the** `eps` **neighborhood** for a point to be considered a Core Point.

> `min_samples = How many points should be there?`

---

4. How `eps` works

Suppose:

`eps = 2`

For every actual data point, DBSCAN looks at other actual data points.

```
A ● -------- B ●
      2
```

If distance ≤ `eps`:

```
B is A's neighbor
```

If:

```
A ● ---------------- B ●
          5
```

and:

`eps = 2`

then:

```
B is NOT A's neighbor
```

So:

> `eps` determines which points are considered neighbors.

---

5. Three types of points

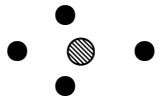
DBSCAN classifies points into:

1. **Core Point**
2. **Border Point**
3. **Noise Point**

5.1 Core Point

A point is a **Core Point** when it has at least `min_samples` points in its `eps` neighborhood.

Example:



If:

`min_samples = 4`

and enough points are within `eps`:

`Number of neighbors ⩾ min_samples`

Then:

= Core Point

Core points are responsible for **creating and expanding clusters**.

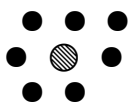
---

6. Border Point

A Border Point:

- does **not** have enough neighbors to be a Core Point
- but is within `eps` of a **Core Point**

Example:



= Core Point

= Border Point

The green point doesn't have enough neighbors itself, but it is close enough to a core point.

Therefore:

> `It belongs to the cluster, but it doesn't expand the cluster.`

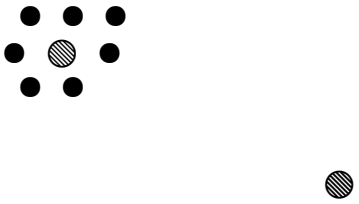
---

7. Noise Point

A point is **Noise** when:

- it doesn't have enough neighbors
- and it is not within `eps` of a Core Point.

Example:



= Noise

In Scikit-learn, noise is represented by:

```
-1
```

---

8. How DBSCAN creates a cluster

This is the most important process.

Suppose we have:

A ● B ● C ● D ● E ●

DBSCAN starts with an **unvisited point**.

Step 1

Take A.

Check A's `eps` neighborhood.

```
A → B, C
```

If A has enough points:

```
A = Core Point
```

A starts a cluster.

Step 2

DBSCAN checks A's neighbors.

For example:

```
A → B, C
```

Now B is checked.

If B is also a Core Point:

```
B → C, D
```

D is added to the same cluster.

Step 3

DBSCAN checks D.

If D is also Core:

```
D → E
```

E is added.

So the cluster keeps expanding.

```
A → B → D → E
```

This continues until there are **no more new points that can expand the cluster**.

---

9. Very important: Does every point get checked?

Yes, DBSCAN eventually considers all points.

But the process isn't simply:

```
A → B → C → D
```

Instead:

```
Choose a point
      ↓
Find its neighbors
      ↓
If Core → expand cluster
      ↓
Check newly discovered points
      ↓
Their neighbors may discover more points
      ↓
Continue until cluster cannot expand
      ↓
Find next unvisited point
```

---

10. What happens if a point doesn't have enough neighbors?

Suppose:

```
min_samples = 4
```

and a point has only:

```
2 neighbors
```

It is **not a Core Point**.

Now DBSCAN asks:

```
Is this point within eps of an existing Core Point?
```

Yes

→ **Border Point**

No

→ **Noise**

Remember:

> Not enough neighbors does NOT automatically mean Noise.

---

11. Important point about checking neighbors

Suppose:

`A → B → C → E`

E is already part of the cluster.

DBSCAN can still check E's own `eps` neighborhood.

For example:

`E → C, F`

If F is within `eps` of E, F can be added to the cluster.

So:

> Every Core Point can help expand the cluster by finding its own neighbors.

This is how DBSCAN discovers connected dense regions.

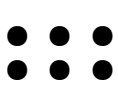
---

12. How does DBSCAN find another cluster?

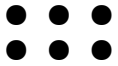
After finishing the first cluster, DBSCAN looks for another **unvisited point**.

Suppose:

`Cluster 1:`



`Cluster 2:`



Noise:



DBSCAN eventually finds a point in the second dense region.

If it has enough neighbors:

```
→ Core Point
→ Start Cluster 2
```

The isolated point may become:

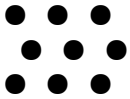
```
→ Noise (-1)
```

---

13. Why is DBSCAN called Density-Based?

Because it focuses on **density**, not centroids.

Dense area:



→ Cluster

Sparse area:





→ Possibly Noise

So:

High density → Cluster

Low density → Noise

14. DBSCAN vs K-Means

K-Means	DBSCAN
Centroid-based	Density-based
Need to specify K	Don't need to specify number of clusters
Usually assigns points to clusters	Can identify noise
Works well for roughly spherical clusters	Can find irregular-shaped clusters
Sensitive to outliers	Handles outliers better
Uses distance to centroids	Uses neighborhood density

15. Why scaling is important

DBSCAN uses **distance**.

Suppose we have:

```
Age        → 20 – 60
Salary     → 20,000 – 200,000
Experience → 0 – 30
```

Salary has much larger values.

Therefore, it can dominate the distance calculation.

So we usually scale the features:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Then apply DBSCAN.

16. How to choose `min_samples`

There is no single perfect value.

A common starting point is related to the number of features.

For example:

Features	Starting value
2	4 – 5
3	6
4	8
5	10

A common rough guideline is:

$$\text{min_samples} \approx 2 \times \text{number of features}$$

But these are only starting values. We can try different values and compare the results.

---

17. How to choose `eps`

A common method is the **K-distance graph**.

Suppose:

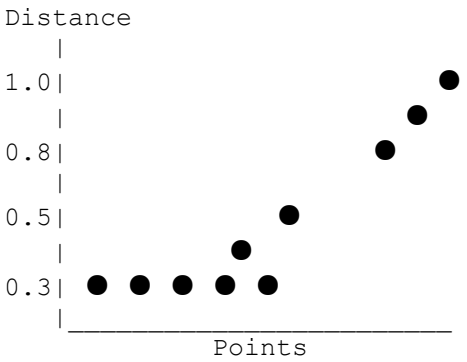
$$\text{min_samples} = 5$$

We calculate the distance from each point to its **5th nearest neighbor**.

Then:

1. Calculate those distances.
2. Sort the distances.
3. Plot them.
4. Look for the **elbow/knee**.
5. Use that region as a candidate `eps`.

Example:



The sharp change is the **elbow/knee**.

That gives us a candidate `eps`.

---

18. What happens if `eps` is too small?

Suppose:

`eps = 0.1`

The neighborhood is very small.

DBSCAN may find very few neighbors.

Result:

`Many points → Noise`

---

19. What happens if `eps` is too large?

Suppose:

`eps = 5`

The neighborhood becomes very large.

DBSCAN may consider many points to be neighbors.

Different clusters can get merged.

So we want a reasonable `eps`.

---

20. DBSCAN implementation

```
from sklearn.cluster import DBSCAN
dbscan = DBSCAN(
```

```
    eps=0.5,
    min_samples=5
)
labels = dbscan.fit_predict(X_scaled)
```

Suppose:

```
labels =
[0, 0, 0, 1, 1, 1, -1, 0, 1]
```

Interpretation:

```
0  → Cluster 0
1  → Cluster 1
-1 → Noise
```

---

21. How to find number of clusters

Unlike K-Means, we don't give:

```
n_clusters = 3
```

DBSCAN discovers the clusters.

You can find the number of clusters from the labels, excluding `-1`.

Conceptually:

```
Unique labels:

0
1
2
-1
```

Means:

```
3 clusters
+
Noise
```

---

22. Evaluation — Silhouette Score

We can use **Silhouette Score** to evaluate clustering.

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X_scaled, labels)
```

General interpretation:

```
Closer to +1 → better-separated clusters
Around 0     → overlapping clusters
Negative     → poor clustering structure
```

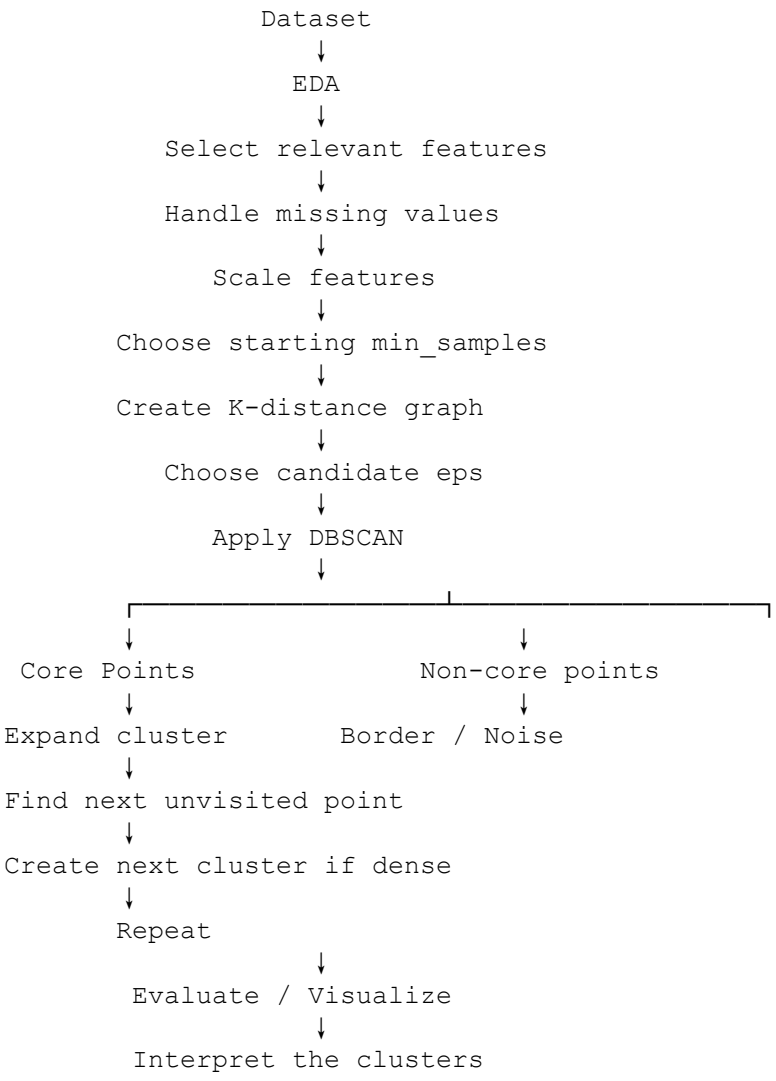
Important

When DBSCAN produces noise (`-1`), it's often better to calculate the silhouette score on the **non-noise points**, provided there are enough clusters/points to make the score meaningful.

---

23. Complete DBSCAN workflow

This is the part you should remember for interviews and projects:



★ Final DBSCAN Cheat Sheet

Definition

> DBSCAN is a density-based clustering algorithm that groups closely packed points and identifies isolated points as noise.

Parameters

```
eps
↓
Maximum neighborhood distance

min_samples
↓
Minimum number of points needed
```

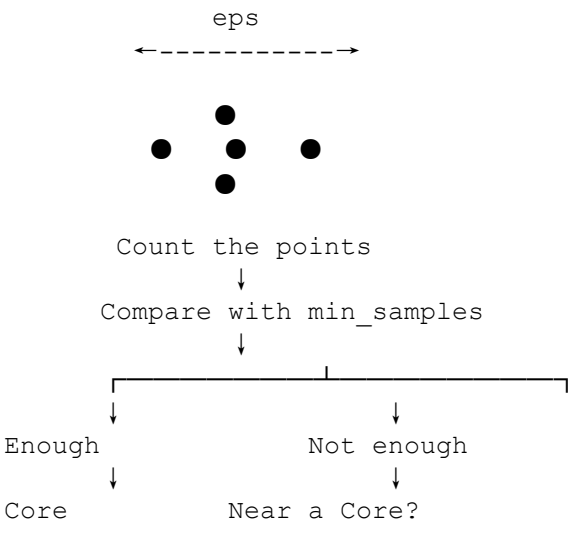
Point types

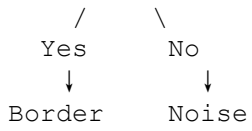
```
Core
↓
Enough neighbors

Border
↓
Not enough neighbors
but near a Core

Noise
↓
Not enough neighbors
and not near a Core
```

Most important concept





One-line memory trick

eps = how far to look.
min_samples = how many points are needed.
Core points expand clusters.
Border points join clusters.
Noise points don't belong to any cluster.